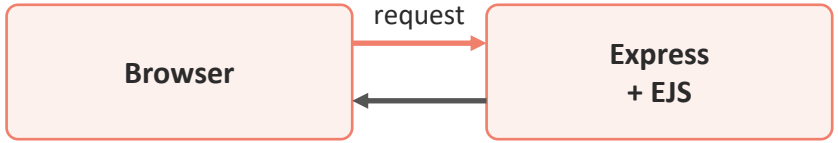
Server-Side vs Client-Side

Express + EJS vs Express + React
Architecture · Request flow · Trade-offs

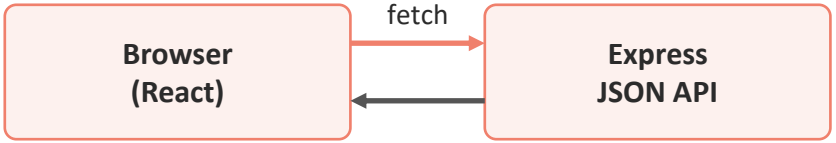
The Core Question: Where Is the HTML Built?

Express + EJS (Server-side)



Complete HTML page returned

Express + React (Client-side)



JSON data returned; browser builds the UI

Server-side rendering (SSR)

Server merges data + template into HTML. Multi-page application.

Client-side rendering (CSR)

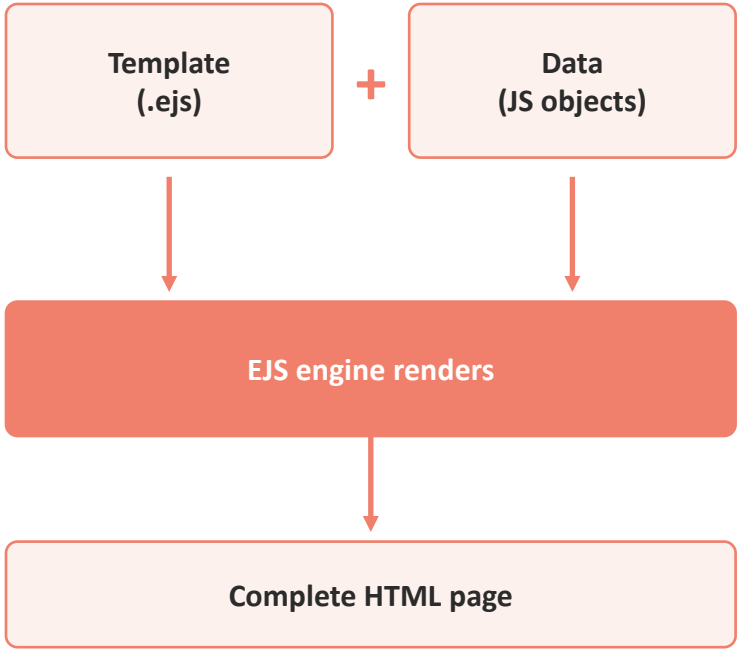
Browser runs JavaScript to build the UI. Single Page Application.

PART 1 — Express + EJS

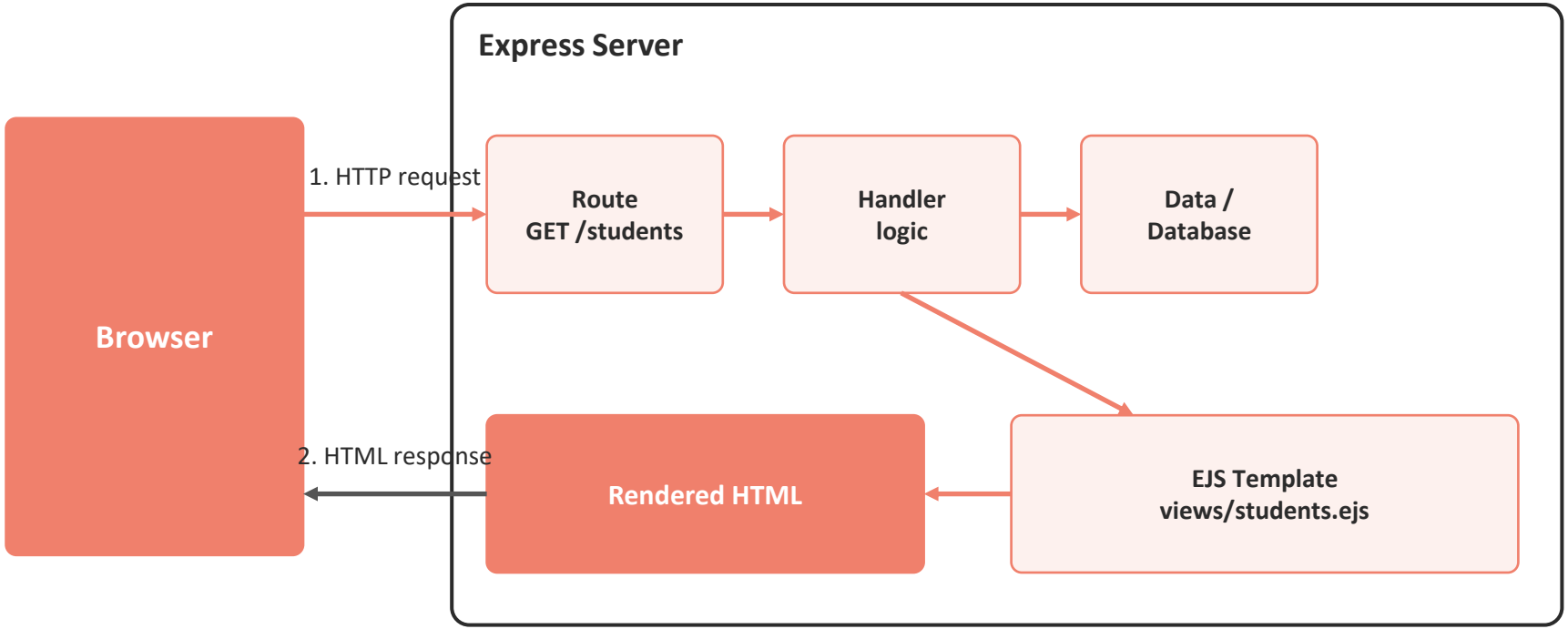
Server-side rendering with templates

Server-Side Rendering with EJS

- EJS = Embedded JavaScript templates
- A template contains HTML with placeholders
- Express fills the template with data on every request
- The browser receives a finished HTML page
- Each link or form submit loads a new page

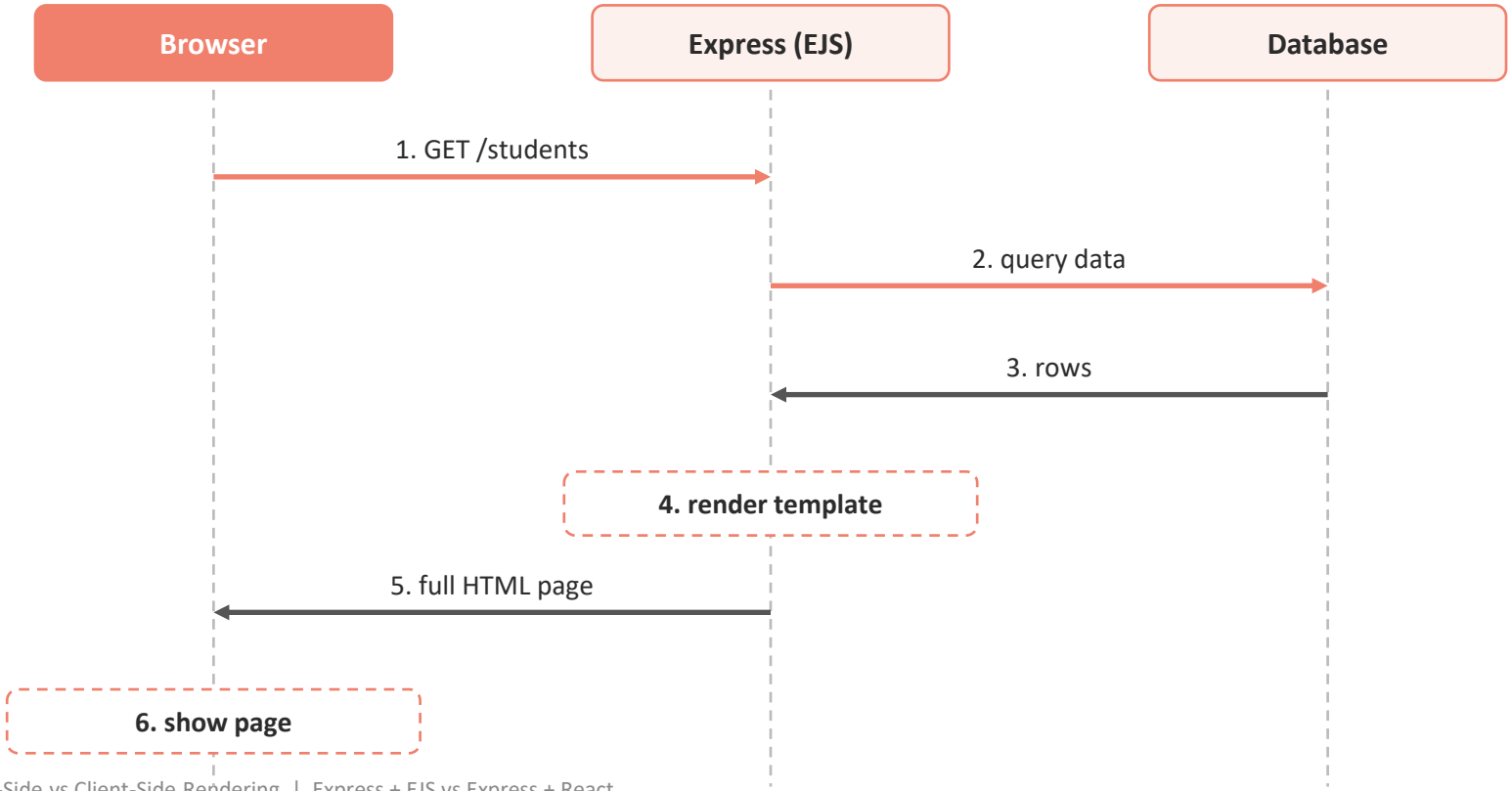


EJS Architecture



Rendering happens entirely on the server.

EJS Request Flow



Navigation in Server-Side Apps



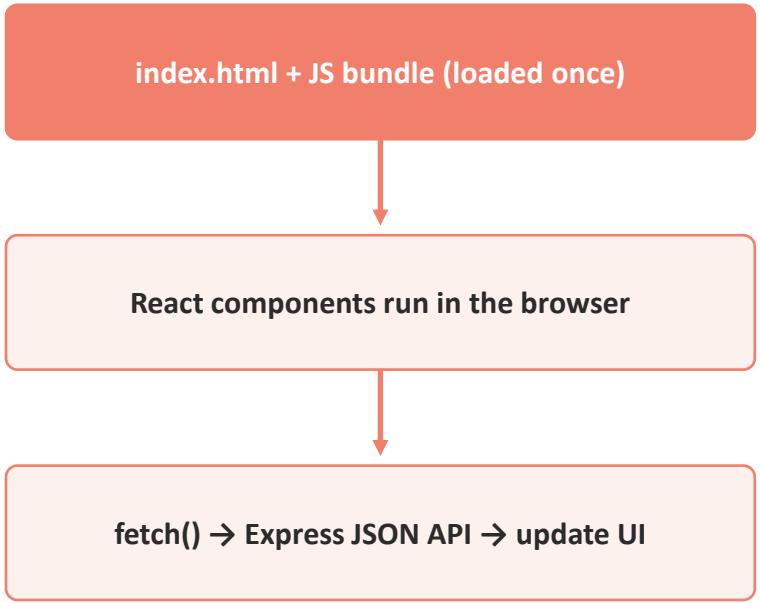
- Every navigation = request + new HTML + full reload
- Simple model, works without client-side JavaScript

PART 2 — Express + React

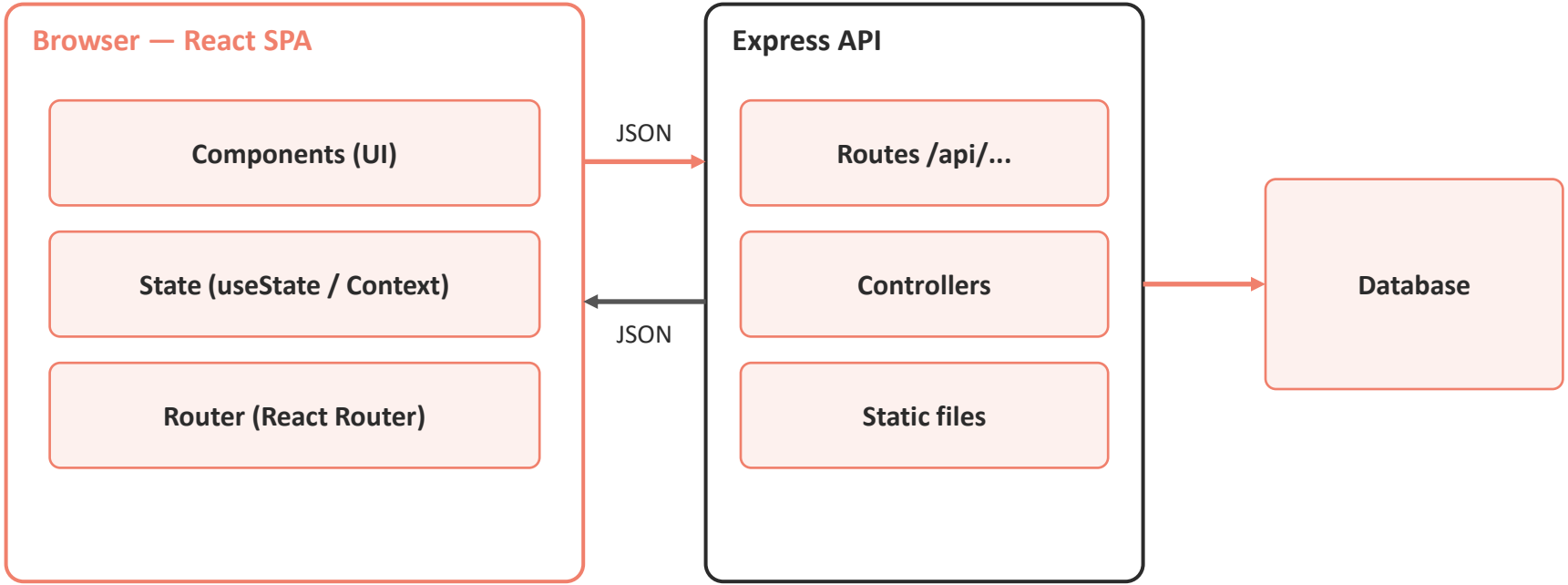
Client-side rendering with a REST API

Client-Side Rendering with React

- React runs in the browser as a Single Page Application
- Express becomes a JSON API (REST backend)
- The browser loads the app once
- Later actions call the API and update only part of the UI
- No full page reloads

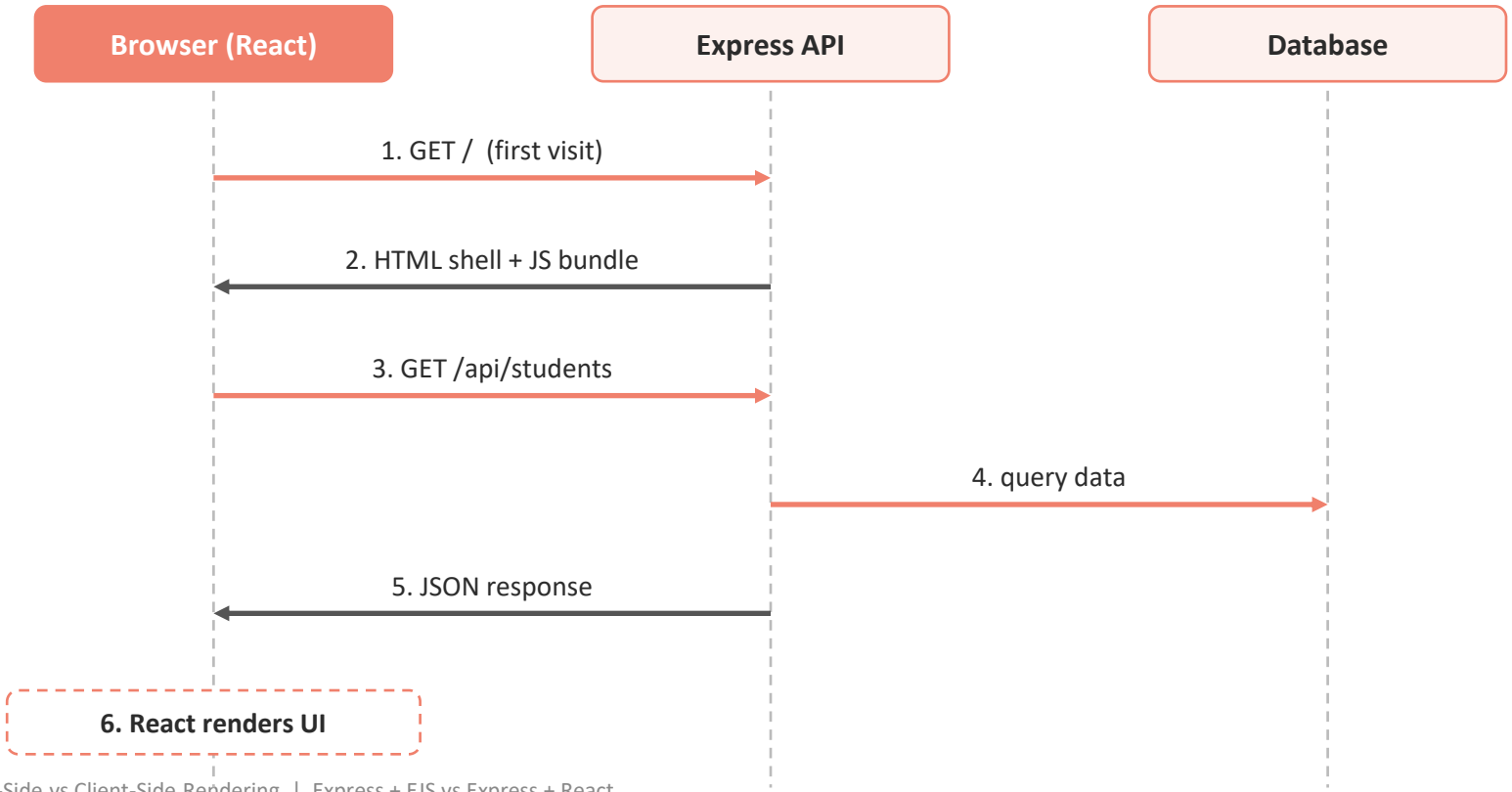


React + Express Architecture



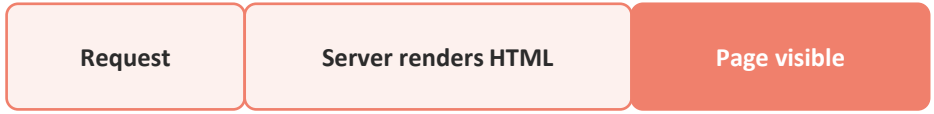
Two separate concerns: UI in the browser, data and logic on the server.

React Request Flow



First Page Load: A Timeline Comparison

EJS (SSR)



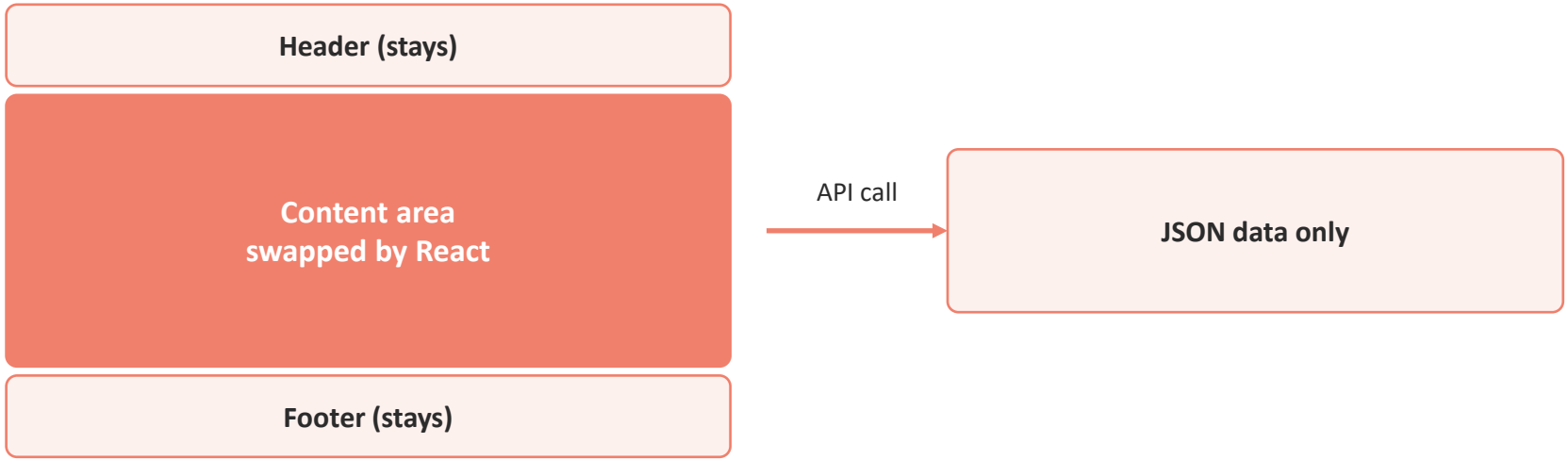
React (CSR)





Illustrative only: React pays up-front for the JS bundle; later interactions are faster.

Navigation in Single Page Apps



- No full reload — only the changed part updates
- Feels like a desktop or mobile app
- Routing is handled in the browser

PART 3 — Comparison

Trade-offs and choosing an approach

Comparison: Development and Operations

Aspect	Express + EJS	Express + React
Learning curve	Low	Higher (JSX, tooling)
Interactivity	Limited	Rich, app-like
State handling	Server (sessions, DB)	Client + server
API reuse (mobile)	Tied to web pages	Same API serves all clients
Team structure	One codebase	Separate frontend and backend
Deployment	One Node app	API + static hosting
XSS safety	<%= %> escapes	JSX escapes by default

SEO: What a Crawler Receives

Express + EJS

```
<h1>Students</h1>
<ul>
  <li>Ravi</li>
  <li>Kiran</li>
</ul>
```

Full content in the first response

Express + React

```
<div id="root"></div>
<script src="bundle.js">
</script>
```

Content appears only after JavaScript runs

Search engines can run JavaScript, but pre-rendered HTML is more reliable and faster to index.

Advantages and Disadvantages

EJS — Advantages

+ Simple, no build step
+ Fast first load, good SEO
+ Works without JavaScript
+ One project, one deployment

React — Advantages

+ Smooth, app-like experience
+ Reusable components, big ecosystem
+ API serves web and mobile
+ Independent frontend / backend

EJS — Disadvantages

− Full reload on each navigation
− Hard to build rich UIs
− Server renders every page
− Less reusable UI than components

React — Disadvantages

− Slower initial load
− SEO needs extra effort
− More tooling (build, routing, CORS)
− Requires JavaScript

When to Use Which

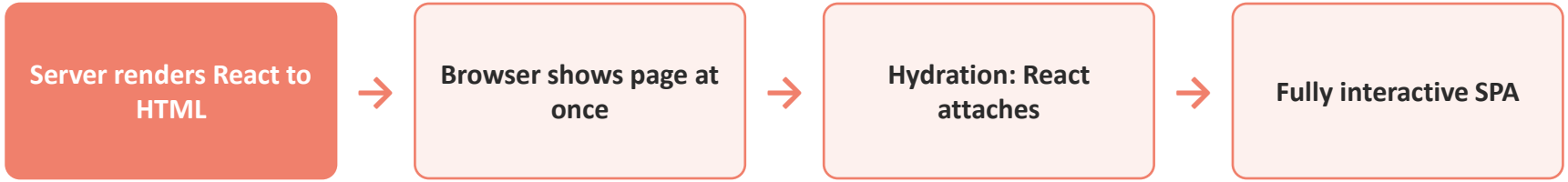
Choose Express + EJS

- Blogs, news, marketing sites
- SEO and first load matter most
- Small apps and prototypes
- Small team, one codebase
- Learning server-side basics

Choose Express + React

- Dashboards and admin tools
- Highly interactive applications
- One API for web and mobile
- Separate frontend / backend teams
- Building modern SPAs (MERN)

A Hybrid: Server-Rendered React (e.g. Next.js)



Approach	HTML built by	Strength
Express + EJS	Server	SEO, simplicity
Express + React	Browser	Interactivity, reusable API
Next.js (SSR/SSG)	Server, then hydrated in browser	Both, with more complexity

Key Points

- Both use Express; the difference is where HTML is built
- EJS: server builds pages, every navigation is a full reload
- React: browser builds the UI, Express is a JSON API
- EJS suits content sites; React suits interactive apps and shared APIs
- Hybrid frameworks combine server rendering with client interactivity